

TRABAJO FINAL

2025

SISTEMAS DISTRIBUIDOS Y PARALELOS

GRACIANO GONZALEZ, FAUSTO	03028/2
MARANZANA, VALENTINA	01917/9

Paralelización del Problema de los N Cuerpos

El problema de los N cuerpos consiste en simular el movimiento de un conjunto de cuerpos que interactúan entre sí usando la ley de gravitación universal de Newton. En cada paso de tiempo, se calculan las fuerzas entre todos los cuerpos y se actualizan sus posiciones y velocidades.

Como el algoritmo tiene una complejidad $O(N^2)$, el tiempo de ejecución se incrementa a medida que aumenta la cantidad de cuerpos. El objetivo de este trabajo es implementar dos versiones paralelas del algoritmo con el fin de poder reducir el tiempo de simulación y permitir una mayor escalabilidad:

- Una solución basada en **Pthreads**, utilizando memoria compartida.
- Una solución **híbrida MPI + Pthreads**, que combina memoria distribuida y compartida.

Solución basada en PTHREADS

ESTRATEGIA DE PARALELIZACIÓN:

La versión con Pthreads adopta un enfoque de paralelización sobre memoria compartida utilizando el paradigma SPMD(Single Program Multiple Data), donde cada hilo (worker) se encarga de calcular las fuerzas ejercidas sobre un subconjunto de cuerpos distribuidos de manera intercalada, es decir, siguiendo una estrategia de stripes, de forma que si hay 4 hilos, el hilo 0 se encarga del cuerpo 0, 4, 8..., el hilo 1 del cuerpo 1, 5, 9..., y así consecutivamente. Esta forma de asignación evita que los hilos se superpongan en su trabajo y garantiza un balance de carga razonable sin requerir divisiones complejas.

Para evitar condiciones de carrera al acumular las fuerzas, se usa una matriz de fuerzas por hilo, donde cada uno guarda sus resultados de manera independiente.

Finalizado el cálculo de las fuerzas, pasamos a la función de mover los cuerpos y siguiendo la estrategia de stripes, cada cuerpo suma las fuerzas que todos los hilos calcularon sobre él para luego actualizar su posición y velocidad.

Entre cada una de estas etapas se utilizan barreras de sincronización para asegurar que todos los hilos hayan completado su tarea antes de avanzar al siguiente paso, garantizando así la coherencia de los datos.

VALIDACIONES:

Para validar la solución obtenida realizamos un programa que calculaba la distancia entre las posiciones finales de los cuerpos de ambas soluciones, mostrando los cuerpos que contaban con una distancia mayor a 10 entre ambas soluciones. Además, este calcula un promedio de las distancias de todos los cuerpos el cual es relativamente bajo como se puede ver en la siguiente imagen.

Pthreads con 512 cuerpos y 4 hilos:

```

/nethome/sdyp13/PRUEBAS_CORREGIDO/directorioSalida/output.txt - sdyp13@163.10.34.49 - Editor - WinSCP
Cuerpo 137: distancia = 55.655895
Cuerpo 206: distancia = 21.367872
Cuerpo 367: distancia = 18.054735
Cuerpo 436: distancia = 29.650141
Promedio de distancia: 0.263799

```

Pthreads con 1024 cuerpos y 4 hilos:

```

/nethome/sdyp13/PRUEBAS_CORREGIDO/directorioSalida/output.txt - sdyp13@163.10.34.49 - Editor - WinSCP
Cuerpo 47: distancia = 34.134197
Cuerpo 111: distancia = 47.268444
Cuerpo 127: distancia = 10.052366
Cuerpo 143: distancia = 25.234228
Cuerpo 223: distancia = 15.566725
Cuerpo 271: distancia = 38.928741
Cuerpo 303: distancia = 19.934162
Cuerpo 351: distancia = 16.874495
Cuerpo 367: distancia = 65.405151
Cuerpo 399: distancia = 28.114937
Cuerpo 479: distancia = 18.567472
Cuerpo 543: distancia = 10.383305
Cuerpo 575: distancia = 24.844673
Cuerpo 639: distancia = 18.438972
Cuerpo 655: distancia = 85.307732
Cuerpo 735: distancia = 52.699100
Cuerpo 815: distancia = 11.619102
Cuerpo 863: distancia = 15.094366
Cuerpo 959: distancia = 14.684532
Promedio de distancia: 0.657728

```

ANÁLISIS DE ESCALABILIDAD:

Tiempos	Tamaño de problema (N)			
	512	1024	2048	4096
Unidades de procesamiento (T)	512	1024	2048	4096
1	11.313149	45.541914	181.831798	727.671162
4	3.668292	13.530420	52.167806	205.950104
8	2.070416	7.104033	26.841845	103.889399

Speedup	Tamaño de problema (N)			
	512	1024	2048	4096
Unidades de procesamiento (T)	512	1024	2048	4096
1	1	1	1	1
4	3.084037	3.365890	3.485517	3.533240
8	5.464191	6.410712	6.774191	7.004286

Eficiencia	Tamaño de problema (N)			
Unidades de procesamiento (T)	512	1024	2048	4096
1	1	1	1	1
4	0.771009	0.841472	0.871379	0.88331
8	0.683023	0.801339	0.846773	0.875535

Basándonos en la definición de programa paralelo escalable: “un programa paralelo es escalable si mantiene constante la eficiencia al aumentar el número de unidades de procesamiento aumentando también el tamaño del problema”, lo que se cumple en nuestro caso, por lo cual el mismo es escalable.

Desde el punto de vista teórico, el análisis de escalabilidad de esta solución se enmarca dentro de la escalabilidad débil, ya que:

- Al aumentar solamente la cantidad de hilos (unidades de procesamiento), manteniendo constante el tamaño del problema, la eficiencia no se mantiene constante. Por ejemplo, para N=512 la eficiencia disminuye de 0.77 (4 hilos) a 0.68 (8 hilos).
- En cambio, cuando se incrementan simultáneamente el tamaño del problema y el número de hilos, la eficiencia mejora o se estabiliza: con N=4096 y T=8, se alcanza una eficiencia de 0.87, lo cual indica que el paralelismo es más aprovechado a medida que aumenta la carga de trabajo.

Este comportamiento se ajusta al modelo de speedup propuesto por Gustafson, el cual plantea que, si aumentamos el tamaño del problema al mismo tiempo que aumentamos los recursos disponibles (procesadores o hilos), la parte paralelizable del algoritmo se vuelve más significativa en relación al total, y por lo tanto se obtiene un mejor aprovechamiento del paralelismo. Podemos ver que se cumple esta ley ya que al aumentar el número de cuerpos se ve un aumento en el número de speedup acercándose al ideal.

Por otro lado, la Ley de Amdahl muestra que, si el tamaño del problema se mantiene fijo, el speedup máximo está limitado por la parte secuencial del programa. Esto significa que, aunque agreguemos más hilos, no vamos a poder reducir indefinidamente el tiempo total porque esa parte secuencial actúa como un cuello de botella. En nuestro caso esta ley se aplica en menor medida ya que al aumentar el número de unidades de procesamiento y manteniendo el mismo número de cuerpos, el speedup si ve un aumento.

En resumen:

- La solución Pthreads no es fuertemente escalable, ya que la eficiencia decae al aumentar T para un N fijo.
- Es débilmente escalable, porque logra mantener o mejorar la eficiencia al incrementar proporcionalmente N y T.

Solución basada en MPI + Pthreads:

ESTRATEGIA DE PARALELIZACIÓN:

La segunda implementación es una versión híbrida que combina paralelismo distribuido y compartido mediante el uso de MPI y Pthreads. El objetivo de esta solución es escalar la aplicación a múltiples nodos de cómputo, manteniendo a la vez la eficiencia dentro de cada nodo.

El total de cuerpos se divide equitativamente entre los procesos MPI, cada uno de los cuales es responsable de un bloque de cuerpos. Dentro de cada proceso, se crean múltiples hilos que paralelizan los cálculos locales siguiendo el mismo esquema de distribución por stripes que en la versión anterior.

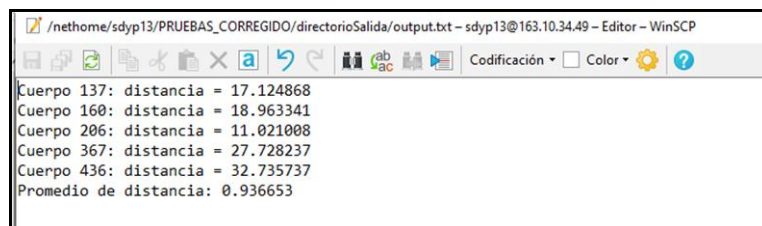
Para esta solución se usó un esquema de comunicación sistólica entre procesos. En cada paso de la simulación, el proceso envía su bloque de cuerpos a los procesos que tienen menor rank, calcula las fuerzas ejercidas por los cuerpos de su propio bloque, y recibe cuerpos de los procesos con mayor rank. A partir de los cuerpos recibidos, cada proceso calcula las fuerzas entre estos y sus cuerpos internos, luego devuelve las fuerzas resultantes. De este modo, se evita centralizar las comunicaciones en un proceso maestro, lo que reduce la congestión y mejora la escalabilidad. Dentro de cada proceso, el cálculo de las fuerzas sigue la misma lógica que en la versión con Pthreads: cada hilo trabaja de forma independiente, evitando conflictos mediante estructuras separadas por hilo.

Finalmente, cada proceso recibe todas las fuerzas que otros calcularon sobre sus cuerpos, las acumula y actualiza las posiciones y velocidades. Todo esto se sincroniza con barreras internas para garantizar que los datos sean consistentes antes de avanzar al siguiente paso de la simulación.

VALIDACIONES:

Al igual que en la solución Pthread para validar la solución obtenida realizamos un programa que calcula la distancia entre las posiciones finales de los cuerpos de ambas soluciones, mostrando los cuerpos que contaban con una distancia mayor a 10 entre ambas soluciones. Y al igual que en la solución de Pthread el promedio de las distancias de todos los cuerpos es relativamente bajo como se puede ver en la siguiente imagen.

Pthreads+MPI con 2 hilos por proceso y 512 cuerpos:



```
/nethome/sdyp13/PRUEBAS_CORREGIDO/directorioSalida/output.txt - sdyp13@163.10.34.49 - Editor - WinSCP
Cuerpo 137: distancia = 17.124868
Cuerpo 160: distancia = 18.963341
Cuerpo 206: distancia = 11.021008
Cuerpo 367: distancia = 27.728237
Cuerpo 436: distancia = 32.735737
Promedio de distancia: 0.936653
```

Pthreads+MPI con 2 hilos por proceso y 1024 cuerpos:

```

/nethome/sdyp13/PRUEBAS_CORREGIDO/directorioSalida/output.txt - sdyp13@163.10.34.49 - Editor - WinSCP
Cuerpo 47: distancia = 19.834379
Cuerpo 79: distancia = 91.157082
Cuerpo 111: distancia = 46.304264
Cuerpo 143: distancia = 22.436787
Cuerpo 207: distancia = 29.010809
Cuerpo 271: distancia = 10.032825
Cuerpo 351: distancia = 16.850336
Cuerpo 367: distancia = 102.800385
Cuerpo 399: distancia = 31.438332
Cuerpo 511: distancia = 20.193144
Cuerpo 575: distancia = 29.410971
Cuerpo 655: distancia = 259.889618
Cuerpo 687: distancia = 13.715799
Cuerpo 735: distancia = 26.179485
Cuerpo 783: distancia = 21.920950
Cuerpo 815: distancia = 17.840746
Cuerpo 863: distancia = 12.352948
Cuerpo 927: distancia = 24.282057
Promedio de distancia: 1.631515

```

ANÁLISIS DE ESCALABILIDAD:

Tiempos	Tamaño de problema(N)			
Unidades de procesamiento (T)	512	1024	2048	4096
1	11.313149	45.541914	181.831798	727.671162
4	5.105821	19.455442	77.132824	307.733788
8	2.809180	10.342842	39.502777	156.084627
16	1.941209	5.934706	21.241508	81.129551

SpeedUp	Tamaño de problema(N)			
Unidades de procesamiento (T)	512	1024	2048	4096
1	1	1	1	1
4	2.215735	2.340831	2.357385	2.364612
8	4.027206	4.403230	4.603013	4.662029
16	5.827888	7.673828	8.560211	8.969249

Eficiencia	Tamaño de problema(N)			
Unidades de procesamiento (T)	512	1024	2048	4096
1	1	1	1	1
4	0.553933	0.585207	0.589346	0.591153
8	0.503400	0.550403	0.575376	0.582753
16	0.364243	0.479614	0.535013	0.560578

El comportamiento observado en los resultados también se ajusta a una escalabilidad débil, debido a que:

- Cuando se incrementa solo el número de unidades de procesamiento (T) y se mantiene constante el tamaño del problema, la eficiencia cae. Por ejemplo, para N=512, la eficiencia desciende desde 0.55 (T=4) hasta 0.36 (T=16).
- Al incrementar tanto T como el tamaño del problema (N), la eficiencia se estabiliza: con N=2048 y T=8, se alcanza una eficiencia de 0.57 y con N=4096 y T=16, se alcanza una eficiencia de 0.56.

Esto sugiere que el algoritmo aprovecha mejor los recursos a medida que la carga de trabajo crece, lo cual está en línea con la Ley de Gustafson, que permite explicar cómo el speedup aumenta con el tamaño del problema en sistemas distribuidos.

La eficiencia no alcanza valores tan altos como en la solución con Pthreads, debido a factores adicionales como overhead de comunicación MPI entre procesos, sincronización entre fases sistólicas o duplicación temporal de datos para el cálculo de fuerzas remotas.

En resumen:

- La solución **MPI + Pthreads es débilmente escalable**.
- Se adapta bien al crecimiento conjunto de carga y recursos, pero no mantiene eficiencia al aumentar solo T con problema fijo.

Conclusión general:

Tanto la versión con Pthreads como la versión híbrida MPI + Pthreads mostraron una mejora significativa en el tiempo de ejecución respecto a la implementación secuencial. La solución con Pthreads escaló correctamente dentro de un único nodo, alcanzando buenos niveles de eficiencia y speedup, especialmente para tamaños de problema grandes. En cuanto a la versión híbrida, logró escalar a múltiples nodos, manteniendo una eficiencia aceptable incluso con 16 unidades de procesamiento, lo que la hace más adecuada para entornos distribuidos. En términos generales, Pthreads ofrece menor complejidad de implementación y buen rendimiento en arquitecturas de memoria compartida, mientras que MPI + Pthreads permite escalar la simulación más allá de un nodo, siendo más eficiente para problemas de mayor tamaño y en sistemas con múltiples máquinas.